

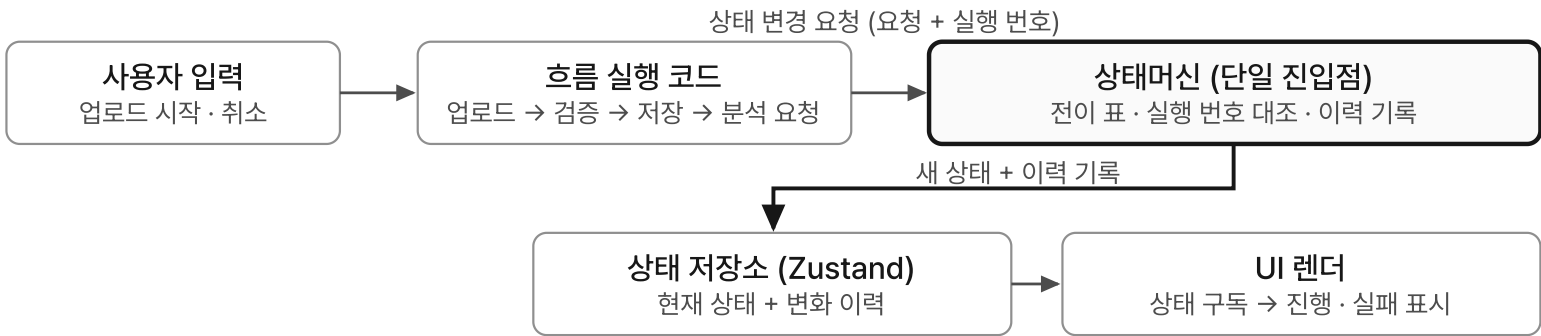
포트폴리오

2024년 7월부터 SAFEAI에서 프론트엔드 개발자로 일하고 있습니다.

SafeSearch의 업로드·분석·취소가 얹힌 흐름을 상태머신 하나로 정리

2025.11 – 2026.07 · 프론트엔드 1인 · React 19, TypeScript, Vite, TanStack Query, Zustand, Leaflet, Vitest

지도 위 여러 CCTV 영상을 오가며 인물의 이동 경로를 추적하는 웹. 업로드 → AI 분석 → 취소·재시작으로 이어지는 긴 흐름에서 화면 상태가 실제 진행과 어긋나지 않도록, 상태 변화를 한곳에 모아 관리하는 장치(상태머신)를 두고 상태 관리를 다시 짰습니다.



모든 상태 변경 요청이 상태머신 한 곳을 통과합니다. 흐름이 화면마다 흩어지지 않고 한 파일에서 읽힙니다.

무엇이 문제였나

1. 상태 변경이 화면마다 흩어져 있었음. 업로드·분석·취소가 얹힌 흐름을 화면마다 흩어진 코드로 관리해, 지금 어느 단계이고 다음에 무엇이 올 수 있는지를 한곳에서 읽을 방법이 없었음
2. 취소하고 다시 시작하면 이전 작업의 응답이 화면을 덮어씀. 취소한 실행과 새 실행을 구분하는 표식이 없어, 뒤늦게 도착한 이전 응답을 최신 상태로 반영해 버림
3. 상태 로직이 화면에 묶여 테스트가 안 됨. 상태 변경이 컴포넌트 안에 있어 화면을 띄우지 않고는 흐름을 검증할 수 없었음

어떻게 풀었나

1. 흐름 전체를 상태머신 하나로 통합하고 전이를 표로 모음. 어느 상태에서 어떤 요청이 오면 다음이 무엇인지를 표로 구조화해 한곳에서 읽고 관리하게 했고, 타입이 모든 상태를 표의 항목으로 강제해 상태를 추가하면 다음 단계 정의를 빠뜨릴 수 없게 함
2. 실행마다 번호를 매겨 이전 실행의 늦은 응답을 걸러냄. 흐름을 시작할 때마다 번호를 매겨 모든 상태 변경 요청에 실어 보내고, 상태머신이 현재 번호와 다른 요청을 걸러내 취소·재시작을 반복해도 화면이 마지막 실행만 따라가게 함
3. 현재 상태와 변화 이력을 함께 기록. 지금 상태와 여기까지 온 경로를 같이 남겨, 문제가 생겼을 때 어느 단계에서 어긋났는지 확인 가능

4. 상태 로직을 화면에서 떼어냄. React·상태 라이브러리에 의존하지 않는 순수 함수로 분리해, 화면을 띄우지 않고 입력과 출력만으로 단위 테스트



취소와 재시작이 겹친 상황. 응답마다 실행 번호를 대조해, 취소된 실행의 늦은 응답이 새 화면을 덮어쓰지 못하게 합니다.

결과

- 1. 업로드·분석 흐름이 상태머신 한 곳으로 수렴. 흐름이 한 파일에서 읽히고, 상태를 추가할 때 다음 단계 정의를 함께 적게 됨
- 2. 취소·재시작을 반복해도 화면이 이전 실행에 덮이지 않음. 실행 번호 대조로 취소 직후 재시작에서 상태가 꼬이던 경로 제거
- 3. 상태 로직을 화면 없이 검증. React 의존 없는 순수 함수로 분리해 Vitest 단위 테스트로 흐름을 고정

Safe Vision 노코드 웹을 React로 다시 만들고 웹·네이티브 통신을 한 곳에 모음

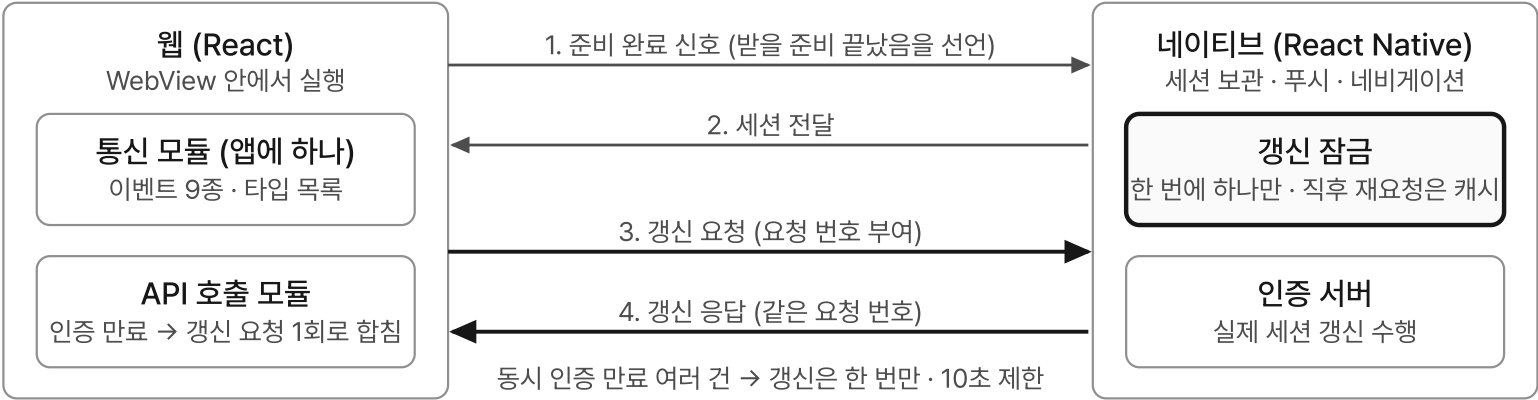
2026.03 - 2026.05 · 프론트엔드 1인 · React 19, TypeScript, Vite, TanStack Query, Zustand, ky, react-native-webview

매장 카메라로 결품(OOS)·매대 변화를 감지하는 AI 비전 서비스의 관리자 웹앱. WeWeb으로 만든 노코드 웹앱을 React로 다시 만들었습니다. 웹과 네이티브 사이의 통신 모듈은 양쪽 모두 직접 작성했습니다. 대형마트 L사 현장 PoC에 투입했고 iOS·Android 앱 스토어에 출시했습니다.

무엇이 문제였나

- 1. 노코드 실행 환경의 성능·유지보수 한계. WeWeb 실행 환경과 편집기 코드가 통째로 전송돼 동일 화면 기준 JS 3,457KB, LCP 19.27s. 초기화·실행 순서를 추적할 수 없어 WebView 연동 디버깅도 어려움
- 2. 웹·네이티브 초기화 순서 미보장. 웹뷰 로드 완료 이벤트로는 웹이 받을 준비를 마쳤는지 알 수 없어, 네이티브가 먼저 세션을 보내면 웹은 로그인 안 된 상태로 진입
- 3. 로그인 갱신 요청과 응답을 짝지을 수 없음. 여러 API가 동시에 인증 만료(401)를 받으면 각각 갱신을 요청하는데 응답에 어느 요청의 것인지 표식이 없고, 응답이 유실되면 무한 대기

어떻게 풀었나



세션 갱신을 네이티브가 도맡는 구조. 웹은 요청과 수신만 하고 실제 갱신은 네이티브의 갱신 잠금 하나가 수행합니다.

- 1. React로 다시 만들고 통신 모듈을 하나로. 웹앱 전체를 React·Vite로 다시 작성하고, 이벤트 9종과 전달 데이터 타입을 목록 한곳에 모아 잘못된 호출을 컴파일 단계에서 차단
- 2. 준비 완료 신호로 순서 보장. 웹이 받을 준비를 마친 직후 준비 완료를 선언해야 네이티브가 세션을 전달하도록 해, 순서를 타이밍이 아닌 수신 측 선언으로 고정
- 3. 요청 번호 대조 + 갱신은 네이티브가 도맡음. 갱신 요청마다 번호를 부여해 자기 요청의 응답만 수락하고, 실제 갱신은 네이티브의 갱신 잠금만 수행. 동시에 여러 번 불러도 한 번만 갱신하고 10초 제한으로 무한 대기 차단

결과

동일 대시보드 기준 WeWeb과 React 재구축 비교. Playwright Chromium, 느린 기기(CPU 4배 저속)·느린 4G 조건, 두 버전을 번갈아 5회 측정.
LCP 실측 구간은 겹치지 않음 (19.2~19.5s vs 3.24~3.34s)

지표	WeWeb	React 재구축	변화
LCP	19.27s	3.29s	-83%
콘텐츠 표시 완료	17.28s	3.29s	-81%
JS 전송량	3,457KB	1,240KB	-64%
JS 힙 사용량	64.9MB	18.4MB	-72%
Long Task 수	8개	2개	-75%

- 1. 초기 진입 시 세션 동기화 순서 보장. 세션 전달이 웹의 준비 완료 이후로 고정, 로그인 안 된 상태로 들어가는 경로 제거
- 2. 인증이 동시에 완료돼도 갱신은 한 번만. 웹에서 한 번 합치고 네이티브에서 다시 잠그는 2단 구조로 중복 갱신 경로 제거